

```

<Dockerfile>+=
# THIS DOCKERFILE IS TANGLED FROM DOCKERFILE.NW. Read the woven version, DOCKERFILE.PDF
# EDIT Dockerfile.nw to make editions.

```

1 Build Dependencies

I have only had success compiling a L^AT_EX document in a Debian 11 environment with T_EXLive 2020, so I utilize the image `texlive-historic-debian:2020`.

```

<Dockerfile>+=
FROM ghcr.io/xu-cheng/texlive-historic-debian:2020 AS texlive
FROM texlive AS noweb-builder
RUN apt-get update && \
    apt-get install -y make gcc git curl wget build-essential && \
    apt-get clean

```

To build Noweb it is necessary to have a recent version of GNU awk, but this is not included in the version of Debian I am using, so I build it from source.

```

<Dockerfile>+=
ENV GAWK_VERSION="5.3.0"
WORKDIR /opt
RUN wget --quiet https://ftp.gnu.org/gnu/gawk/gawk-${GAWK_VERSION}.tar.gz && \
    tar -xzf gawk-${GAWK_VERSION}.tar.gz
WORKDIR /opt/gawk-${GAWK_VERSION}
RUN ./configure && make && make install
RUN update-alternatives --remove-all awk || true && \
    update-alternatives --install /usr/bin/awk awk /usr/local/bin/gawk 10

```

1.1 Environment Variables

```

<Dockerfile>+=
# Set environment variables for installation locations
ENV NOWEB_BIN=/usr/local/bin \
    NOWEB_LIB=/usr/local/share/noweb \
    NOWEB_MAN=/usr/local/share/man \
    NOWEB_TEXINPUTS=/usr/local/share/texmf/tex/latex/knoweb \
    KNOWEB_SRC=/opt/knoweb \
    KNOWEB_STYLE_DEST=/usr/local/share/texmf/tex/latex/knoweb \
    TEXINPUTS="/usr/local/share/texmf/tex/latex/:" \
    NOWEB_SRC=/opt/noweb/src

```

1.2 Build and Install Noweb

The next step is to build Noweb from my fork, which can now possible in this Debian base-image with GNU awk built from sources.

Noweb will be installed into `NOWEB_BIN`; Noweb is built using shell arguments instead of editing the Makefile (as Ramsey recommends).

I intend to improve my contributions to Noweb before making a pull request, but before then I intend to format my changes as a patch to the original Noweb so that I can merely distribute a patch for Noweb alongside the vendored original sources in a Git submodule.

It's not unreasonable to say, "Building this software requires noweb with these patches applied". Perfectly fine. The patches improve the Aho-Corasick recognizer by allowing it to detect kebab-casing in LISP identifiers.

```

(Dockerfile)+≡
WORKDIR /opt
RUN git clone https://github.com/bryce-carson/noweb.git
WORKDIR ${NOWEB_SRC}
RUN ./awkgname gawk

## Dry run, with detail, to help debugging.
ARG GITHUB_CI_BUILD="false"
RUN <<ETX
if [ "$GITHUB_CI_BUILD" = "true" ]; then
    echo "Running in GitHub CI!";
    touch ${NOWEB_SRC}/c/*.c ${NOWEB_SRC}/c/*.h;
    cd c && make -j1 -nd markup;
    make -j1 -n -d CC="gcc" CFLAGS="-Wall" \
    BIN=$NOWEB_BIN \
    LIB=$NOWEB_LIB \
    MAN=$NOWEB_MAN \
    TEXINPUTS=$NOWEB_TEXINPUTS \
    all;
fi
ETX

RUN <<ETX
make -j1 CC="gcc" CFLAGS="-Wall" \
BIN=$NOWEB_BIN \
LIB=$NOWEB_LIB \
MAN=$NOWEB_MAN \
TEXINPUTS=$NOWEB_TEXINPUTS \
all install
ETX

```

1.3 Build and install knoweb

knoweb is a L^AT_EX style file for Noweb with improvements over the upstream style file.

The next step is to clone, build, and install `knoweb.sty` and `autodefs.elisp` from `JoeRiel/knoweb`; also, refresh the T_EX file database so T_EXLive finds `knoweb.sty`.

Formerly, the entire build and installation process for knoweb was contained in the `Dockerfile`, but that prevented me from using rules in the `Makefile` to ensure I had knoweb successfully built for my actual development machine; it's not pleasant to work in the terminal of a container.

```
<Dockerfile>+≡
    WORKDIR /opt
    RUN make knoweb
```

1.4 Make the WHYSEP_{PDF}

Semifinally, it is time to compile the PDF for WHYSE, which will also build the GNU EMACS LISP package.

```
<Dockerfile>+≡
    FROM noweb-builder AS whyse-builder
    COPY . /workspace
    WORKDIR /workspace
    RUN make NOWEB_LIB="${NOWEB_LIB}" SRC="/workspace/src" BUILD="/workspace/build" TEST=","
```

Finally, artifacts are placed in the root `/build` directory.

```
<Dockerfile>+≡
    FROM debian:stable-slim AS output
    COPY --from=whyse-builder /workspace/build /build
    VOLUME ["/build"]
```